

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to VTU, Belagavi, Approved by AICTE & ISO 9001:2008 Certified)

Accredited by National Assessment & Accreditation Council (NAAC) with 'A' grade,
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-111



Report

On

“ SMART CAMPUS INFORMATION SYSTEM APPLICATION”

Submitted by

Anushka Yadav [1DS25IC011]

Second Semester B.E

(CSE) 2025-2026

Under the guidance of

Alka Rani

Assistant Professor

Dept. of CSE

DSCE, Bangalore

Department of Computer Science and Engineering

Bangalore-780111

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
DAYANANDA SAGAR COLLEGE OF ENGINEERING
ShavigeMalleshwaraHills,KumaraswamyLayout,Bangalore - 560111
Department of ComputerScience & Engineering



CERTIFICATE

This is to certify that the application entitled “**SMART CAMPUS INFORMATION SYSTEM APPLICATION**” is a bonafide work carried out by **Anushka Yadav [1DS25CD011]** during the **Second Semester Python Programming Laboratory Sessions, Bachelor of Engineering in Computer Science and Engineering** under **Visvesvaraya Technological University, Belgaum** in the academic year **2025–26**.

Ms. Alka Rani

Assistant Professor,

Department of CSE,

DSCE

Signature.....

Dr.Ramesh Babu

Vice principal

Department of CSE,

DSCE

Signature.....

Nameof the Examiners:

1.

2.

Signature with date:

.....

.....

Application – Smart Campus Information System

Dayananda Sagar College of Engineering plans to develop a simple Python-based Smart Campus Information System to manage student information, academic records, and performance analysis. At present, several administrative tasks such as student registration, course enrollment, record maintenance, searching student data, fee calculation, and performance analysis are often handled manually. These manual processes can be time-consuming, inefficient, and prone to errors.

To improve efficiency and accuracy, the institution intends to implement a basic software system that enables administrative staff to perform various academic management tasks digitally. The system will allow administrators to register students, manage course enrollments, maintain academic records, search and sort student data, calculate fees, store and retrieve records from files, scan directories for project files, and analyze student performance using data analytics tools.

The Smart Campus Information System will be developed gradually through Python laboratory experiments. Each experiment focuses on implementing a specific functional module, and together these modules form a complete integrated application. The final system will consist of several modules such as Student Registration, Course Enrollment Management, Student Records Management, Search and Sorting, Fee Calculation, File Management, Directory Scanning, and Performance Analytics, all integrated through a Main System Application Dashboard.

Students will develop these modules week by week, and in the final weeks they will integrate all modules to demonstrate the complete working of the Smart Campus Information System.

Overview of the Application:



Smart Campus Information System

- student_registration.py (Lab 1)
- course_enrollment.py (Lab 2)
- student_records.py (Lab 3)
- search_sort_students.py (Lab 4)
- fee_calculation.py (Lab 5)
- file_manager.py (Lab 6)
- directory_scanner.py (Lab 7)
- performance_analytics.py (Lab 8)
- main_application.py (Lab 9&10)

Smart Campus Information System (Mini Project Integration) : Demonstrate a complete Python application for the Smart Campus Information System by integrating all the modules developed from Lab 1 to Lab 8.

The final program should demonstrate the working of all components including:

- Student registration and grade evaluation
- Course enrollment management
- Student record storage and management
- Searching and sorting student data
- Fee calculation using functions
- File-based academic record management
- Directory scanning with exception handling
- Student performance analytics using NumPy, Pandas, and Matplotlib

INDEX

1. Introduction
2. Objectives
3. Explanation of Implementation
4. Python Program Code
5. Sample Outputs
6. Overview Flowchart
7. Conclusion

INTRODUCTION

The Smart Campus Information System is a Python-based mini project developed to automate and simplify academic management activities in a college campus . The system integrates multiple modules such as student registration, course enrollment, record management, searching and sorting, fee calculation, file handling, directory scanning, and performance analytics.

The project demonstrates the practical implementation of Python concepts including:

- Conditional statements
- Loops
- Functions
- Lists, Dictionaries, and Sets
- Searching and Sorting algorithms
- File handling
- Exception handling
- NumPy, Pandas, and Matplotlib libraries

The system helps in reducing manual work and improves efficiency in handling student-related data.

OBJECTIVES

The objectives of the Smart Campus Information System are:

1. To manage student academic information efficiently.
2. To implement Python programming concepts in a real-world application.
3. To automate student registration and grading processes.
4. To perform searching, sorting, and data analysis operations.
5. To demonstrate file handling and exception handling techniques.
6. To analyze student performance using data analytics libraries.

EXPLANATION OF IMPLEMENTATION

- The Smart Campus Information System is a Python-based application developed for Dayananda Sagar College of Engineering to digitize and simplify manual campus administration tasks such as student registration, fee calculation, and academic performance analysis. The system integrates multiple modules into a single Main Dashboard for efficient management.

Key Components of the system

1. Student Registration & Course Enrollment

- Uses conditional statements to evaluate marks and assign grades with performance remarks.
- Implements loops and validation checks to manage course enrollment with a limit of 5 courses per student.

2. Student Data Management

- Stores student details using lists and dictionaries.
- Uses sets to analyze common participants in campus events.
- Applies sorting and binary search algorithms for faster student record retrieval.

3. Fee & File Management

- Calculates fees using functions with optional parameters for hostel and transport charges.
- Uses file handling to store and retrieve academic records permanently.
- Generates summaries such as class averages and top-performing students.

4. Error Handling & Directory Scanning

- Uses Python's os module for directory management.
- Implements exception handling and custom exceptions to prevent crashes from invalid files or folders.

5. Data Analytics & Visualization

- Uses Pandas and NumPy for statistical analysis like mean, median, and standard deviation.
- Generates Matplotlib charts to visualize student and subject performance.

Overall, the project demonstrates the integration of Python programming concepts, data structures, file handling, algorithms, and data analytics to create a complete campus management solution.

PYTHON PROGRAM CODE

```
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# GLOBAL DATA STORAGE
records = []

# MODULE 1 & 3 – REGISTRATION, EVALUATION & DISPLAY
def register_student():
    name, sid = input("Name: "), input("ID: ")
    try:
        marks = int(input("Marks (0-100): "))
    except ValueError:
        print("Invalid input!"); return

    # Compact Grade Evaluation mapping
    for limit, (g, r) in {90:("O","Outstanding"), 80:("A+","Excellent"), 70:("A","Very Good"),
        60:("B","Good"), 50:("C","Average")}.items():
        if marks >= limit: grade, remark = g, r; break
    else: grade, remark = "F", "Needs Improvement"

    records.append({"name": name, "id": sid, "marks": marks, "grade": grade, "courses": []})
    print(f"\n===== REPORT =====\nName: {name}\nID: {sid}\nMarks: {marks}\nGrade: {grade}\nRemark: {remark}\n")

def display_records():
    if not records: print("No records available!\n"); return
    print("\n===== STUDENT RECORDS =====")
    for s in records:
        print(f'ID: {s["id"]} | Name: {s["name"]} | Marks: {s["marks"]} | Grade: {s["grade"]}')
        print(f'Courses: {', '.join(s["courses"]) if s["courses"] else 'None'}\n-----")

# MODULE 2 – COURSE ENROLLMENT
def enroll_courses():
    sid = input("Enter student ID: ")
    student = next((s for s in records if s["id"] == sid), None)
    if not student: print("Student not found!\n"); return

    print("\nAdd max 5 courses (type 'done' to stop)")
    while len(student["courses"]) < 5:
        course = input("Course name: ").strip()
        if course.lower() == "done": break
        if course: student["courses"].append(course)
    print("Courses updated!\n")
```

MODULE 4 – SORTING & SEARCHING (Optimized built-ins)

```
def sort_and_search():
    if not records: print("No records available!\n"); return
    ids = sorted([s["id"] for s in records])
    print(f"\nSorted Student IDs: {ids}")

    key = input("\nEnter ID to search: ")
    if key in ids:
        print(f"Student ID found at sorted index {ids.index(key)}\n")
    else:
        print("Student ID not found!\n")
```

MODULE 5 – FEE CALCULATION

```
def fee_menu():
    try:
        h = int(input("Hostel fee (0 if none): "))
        t = int(input("Transport fee (0 if none): "))
    except ValueError: print("Invalid input!"); return
    print(f"\n===== FEE DETAILS =====\nTuition: 50000\nHostel: {h}\nTransport: {t}\nTotal: {50000 + h + t}\n")
```

MODULE 6 – FILE HANDLING

```
def file_handling():
    fn = "student_records.txt"
    with open(fn, "w") as f:
        f.write("ID,Name,Marks,Grade\n")
        f.writelines(f'{s["id"]},{s["name"]},{s["marks"]},{s["grade"]}\n' for s in records)
    print("Records written! \n\n===== FILE CONTENT =====")
    with open(fn, "r") as f: print(f.read())
```

MODULE 7 – DIRECTORY SCANNING

```
class EmptyFolderError(Exception): pass

def scan_directory():
    path = input("Enter directory path: ")
    try:
        if not os.path.exists(path): raise FileNotFoundError("Path does not exist!")
        print("\nScanning...\n")
        for root, dirs, files in os.walk(path):
            level = root.replace(path, "").count(os.sep)
            print(f'{' ' * 4 * level} {os.path.basename(root)}\n')
            for file in files: print(f'{' ' * 4 * (level + 1)} {file}\n')
            if not files and not dirs: raise EmptyFolderError(f"Empty folder: {root}")
    except Exception as e: print("Error:", e)
```

MODULE 8 – PERFORMANCE ANALYSIS

```
def performance_analysis():
    if not records: print("No records available!\n"); return
    df = pd.DataFrame([{"Name": s["name"], "Marks": s["marks"]} for s in records])
    print(f"\n===== DATA =====\n{df}\n\n===== SUMMARY =====\n{df.describe()}")

    top = df.loc[df["Marks"].idxmax()]
    print(f"\nAverage Marks: {np.mean(df['Marks'])}\nTop Performer: {top['Name']} ({top['Marks']})\n")

    plt.figure(figsize=(6, 4)); plt.bar(df["Name"], df["Marks"])
    plt.title("Performance"); plt.xlabel("Names"); plt.ylabel("Marks"); plt.show()
```

MAIN DASHBOARD

```
def main():
    menu = {"1": register_student, "2": enroll_courses, "3": display_records, "4": sort_and_search,
           "5": fee_menu, "6": file_handling, "7": scan_directory, "8": performance_analysis}
    while True:
        print("="*38 + "\n SMART CAMPUS INFORMATION SYSTEM \n" + "="*38)
        for k, v in menu.items(): print(f'{k}. {v.__name__.replace('_', ' ').title()}')
        choice = input("9. Exit\n" + "="*38 + "\nChoice: ")
        if choice == "9": print("\nThank You!"); break
        menu.get(choice, lambda: print("Invalid choice!\n"))()

if __name__ == "__main__":
    main()
```

OUTPUT

Main Menu

```
=====
SMART CAMPUS INFORMATION SYSTEM
=====
1. Register Student
2. Enroll Courses
3. Display Records
4. Sort And Search
5. Fee Menu
6. File Handling
7. Scan Directory
8. Performance Analysis
9. Exit
=====
Choice:
```

Scenario 1: Selecting Option 1

```
Enter your choice: 1
Enter student name: Tanu
Enter student ID: 2
Enter marks (0-100): 87

===== STUDENT REPORT =====
Name      : Tanu
ID         : 2
Marks      : 87
Grade      : A+
Remark     : Excellent
```

```
Enter your choice: 1
Enter student name: Shreya
Enter student ID: 1
Enter marks (0-100): 78

===== STUDENT REPORT =====
Name       : Shreya
ID          : 1
Marks       : 78
Grade       : A
Remark      : Very Good
```

Scenario 2: Selecting Option 2

```
Enter your choice: 2
Enter student ID: 2

Add maximum 5 courses
Type 'done' to stop

Enter course name: Maths
Enter course name: done
Courses enrolled successfully!
```

Scenario 3: Selecting Option 3

```
===== STUDENT RECORDS =====
ID      : 1
Name     : Shreya
Marks    : 78
Grade    : A
Courses  : Maths
-----
ID      : 2
Name     : Tanu
Marks    : 87
Grade    : A+
Courses  : Maths
-----
```

Scenario 4: Selecting Option 4

```
Sorted Student IDs:
['1', '2']

Enter ID to search: 2
Student ID found at position 1
```

Scenario 5: Selecting Option 5

```
Enter hostel fee (0 if none): 50000
Enter transport fee (0 if none): 0

===== FEE DETAILS =====
Tuition Fee : 50000
Hostel Fee   : 50000
Transport    : 0
Total Fee    : 100000
```

Scenario 6: Selecting Option 6

```
Records written successfully!

===== FILE CONTENT =====
ID,Name,Marks,Grade
1,Shreya,78,A
2,Tanu,87,A+
```

Scenario 7: Selecting Option 7

```
Enter directory path: C:\Users\Anushka\OneDrive\Desktop\New folder
Scanning Directory...

New folder/
  22ESC142- Module -3 - DC Generator_motor_notes SJ and SA.pdf
  2nd module python.docx
  chem mod 2.pdf
  chem mod 3 .pdf
  mod 3python.docx
```

Scenario 8: Selecting Option 8

```
===== STUDENT PERFORMANCE DATA =====
```

	Name	Marks
0	Shreya	78
1	Tanu	87

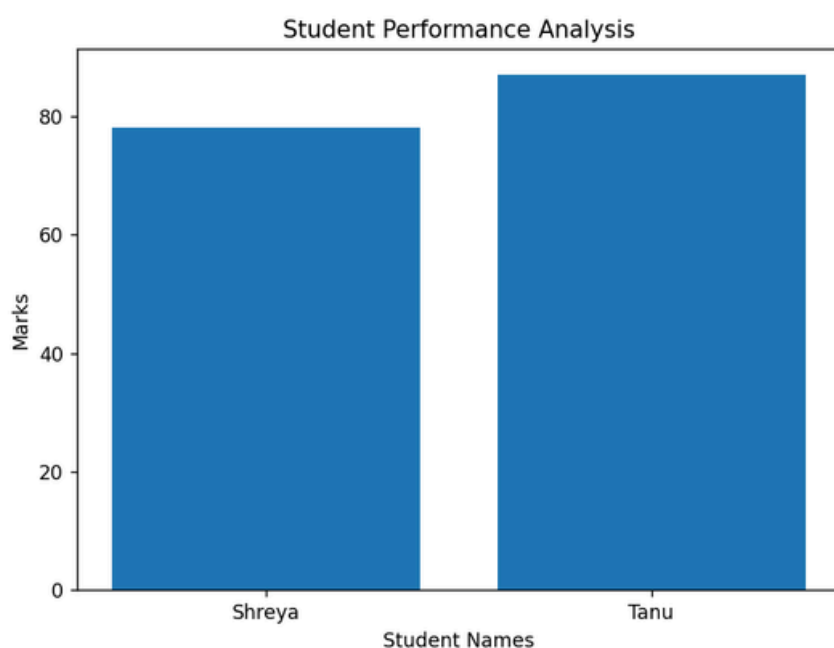
```
===== STATISTICAL SUMMARY =====
```

	Marks
count	2.000000
mean	82.500000
std	6.363961
min	78.000000
25%	80.250000
50%	82.500000
75%	84.750000
max	87.000000

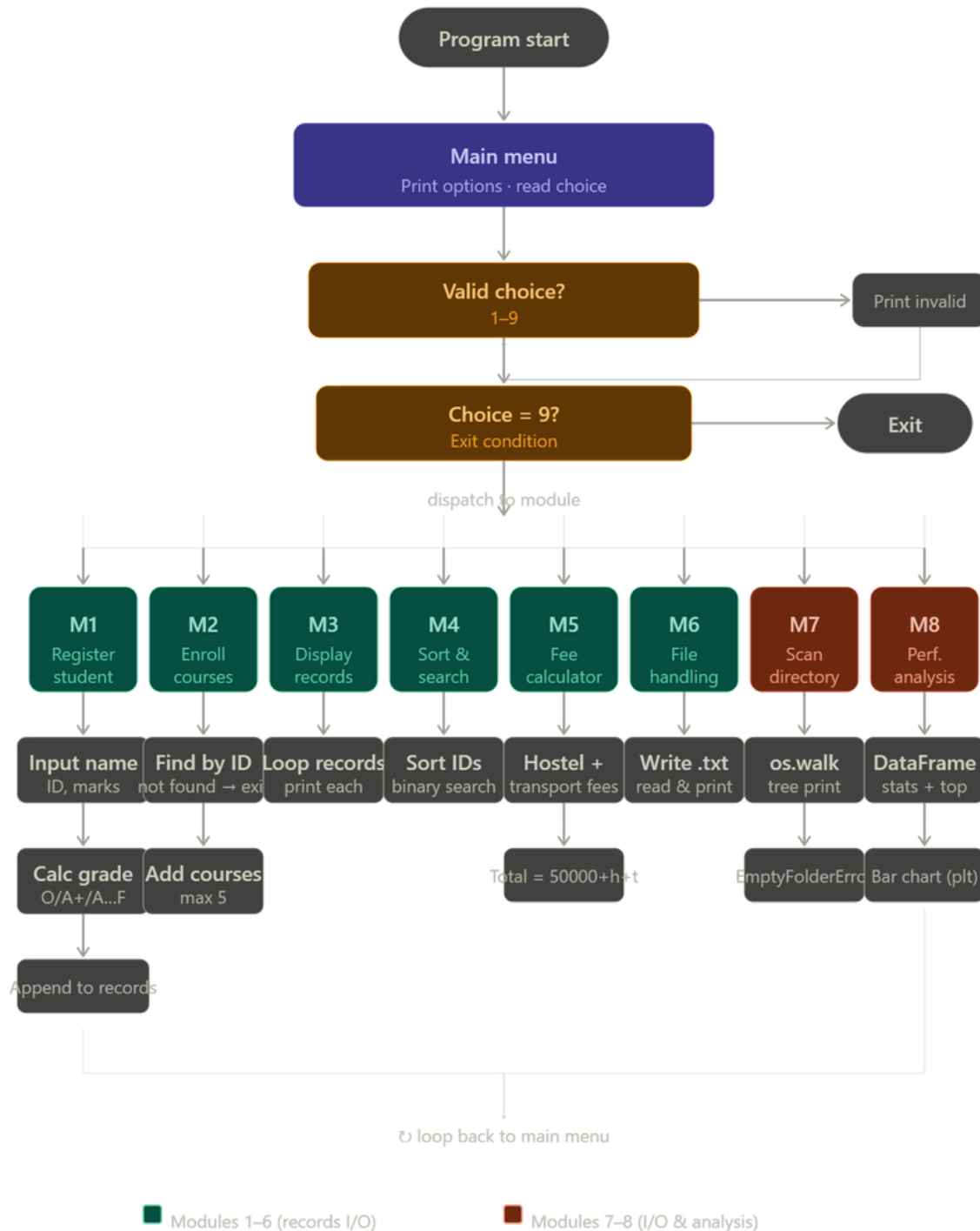
Average Marks : 82.5

Top Performer : Tanu

Top Marks : 87



OVERVIEW FLOWCHART



CONCLUSION

The Smart Campus Information System successfully demonstrates the integration of multiple Python programming concepts into a single application. The project provides practical exposure to problem-solving, file handling, data analysis, searching and sorting techniques, exception handling, and visualization tools. This mini project helped in understanding real-world implementation of Python programming for academic management systems.